NEXORA TECHNOLOGIES PVT. LTD.

Product, Engineering & Software Development Handbook

Document ID: NXR-TECH-002
Version: 1.0
Effective Date: 01 April 2026
Review Date: 31 March 2027
Document Owner: Technology Organization
Approved By: Vikram Rao, Chief Technology Officer
Information Classification: Internal

---

1. Purpose, Scope & Engineering Philosophy

a. Purpose

This handbook defines the standards, operating practices and responsibilities used by Nexora Technologies for product development, software engineering and technology delivery.

It establishes a common engineering framework while allowing individual teams to select implementation approaches appropriate to their products and customers.

The handbook covers:

- Product engineering
- Software architecture
- Requirements
- Product planning
- Development
- Source control
- Code review
- Testing
- Quality engineering
- API development
- Database engineering
- DevOps
- CI/CD
- Release engineering
- Technical documentation
- Open-source software
- Technical debt

- Engineering metrics
- Software maintenance
- Product retirement

It applies to employees, contractors and approved consultants who design, build, test, deploy or maintain Nexora software.

b. Engineering Philosophy

Nexora engineering teams follow six principles.

I. Solve the right problem

Technology should address a clearly understood business, customer or operational need.

II. Prefer simple solutions

Complexity must be justified by measurable requirements.

III. Build quality into the process

Quality is not a final testing activity. It is a responsibility throughout the development lifecycle.

IV. Protect information by design

Security, privacy and appropriate data handling must be considered during architecture and implementation.

V. Automate repeatable work

Builds, tests, deployments, infrastructure and operational checks should be automated where practical.

VI. Own the outcome

Teams remain accountable for the software they put into production.

---

2. Engineering Organization

a. Technology Organization

The Technology organization consists of several closely connected functions.

Function	Primary Responsibility
Product Engineering	Software products and applications
Platform Engineering	Shared platforms and developer capabilities
Data Engineering	Data platforms and pipelines
AI Engineering	AI and ML-powered systems
Cloud & Infrastructure	Cloud platforms and infrastructure

Function	Primary Responsibility
Quality Engineering	Software quality and testing
Architecture	Technical direction and architecture
Security Engineering	Application and platform security
DevOps / SRE	Delivery, reliability and operations

The organization operates across Bengaluru, Pune, Hyderabad and London.

b. Engineering Career Levels

The engineering career framework is aligned with the company-wide job-level structure.

Level	Typical Engineering Role
L1	Graduate Engineer
L2	Software Engineer
L3	Software Engineer / Data Engineer
L4	Senior Software Engineer
L5	Lead Engineer / Specialist
L6	Engineering Manager / Principal Engineer
L7	Senior Engineering Manager / Senior Principal
L8	Engineering Director / Architect
L9	Executive Technology Leadership

Employees may progress through either management or specialist technical paths.

c. Engineering Manager

Engineering managers are accountable for:

- Team development
- Delivery
- Staffing
- Performance
- Engineering health
- Risk management
- Stakeholder communication

They are not expected to make every technical decision personally.

d. Technical Lead

Technical leads are responsible for:

- Technical direction
- Architecture within their scope
- Code quality
- Technical risk
- Mentoring
- Engineering decisions

---

3. Product Development Model

a. Product Lifecycle

Nexora uses the following general product lifecycle:

```
Opportunity
    ↓
Discovery
    ↓
Problem Definition
    ↓
Product Requirements
    ↓
Prioritization
    ↓
Architecture & Design
    ↓
Development
    ↓
Quality Validation
    ↓
Release
    ↓
Adoption & Monitoring
    ↓
Continuous Improvement
    ↓
Retirement
```

The lifecycle is iterative.

Teams may return to earlier stages when new information changes the product direction.

b. Product Discovery

Discovery should establish:

- Customer problem
- Target user

- Business value
- Existing alternatives
- Constraints
- Risks
- Success measures

Discovery should not automatically result in a software project.

A problem may instead be solved through:

- Process improvement
- Configuration
- Automation
- Existing platform capability
- Training
- Customer education

c. Product Success Metrics

Products should define measurable success indicators.

Examples include:

- Active users
- Conversion
- Adoption
- Retention
- Customer satisfaction
- Revenue
- Cost reduction
- Processing time
- Error reduction

A product initiative should normally have between **3 and 7 primary success metrics**.

---

4. Product Requirements

a. Requirements Structure

Product requirements should explain:

1. Problem
2. User
3. Desired outcome
4. Functional requirements
5. Non-functional requirements

6. Constraints
7. Acceptance criteria

b. Functional Requirements

Functional requirements describe system behavior.

Example:

> *Authorized users must be able to export customer transaction records for a selected date range.*

c. Non-Functional Requirements

Non-functional requirements may cover:

- Performance
- Availability
- Security
- Scalability
- Accessibility
- Reliability
- Auditability
- Cost
- Maintainability

Example:

> *The reporting API should return a standard report within 5 seconds for at least 95% of requests under the agreed production workload.*

d. Acceptance Criteria

Acceptance criteria should be observable and testable.

Example:

```
Given an authorized finance user
When a report is requested for January 2026
Then the system returns all approved transactions
for the selected period.
```

e. Requirement Changes

Requirements may change during development.

Material changes should be documented when they affect:

- Scope
- Schedule
- Budget
- Architecture
- Security
- Customer commitments

---

5. Product Prioritization

a. Priority Categories

Product teams may use:

Priority	Definition
P0	Critical / mandatory
P1	High-value
P2	Normal
P3	Optional

P0 should be reserved for genuinely critical requirements.

b. Prioritization Factors

Prioritization should consider:

- Customer impact
- Revenue impact
- Strategic value
- Risk reduction
- Engineering effort
- Dependencies
- Regulatory obligations
- Operational cost

c. Engineering Effort

Teams should estimate work before committing to delivery.

Estimation may use:

- Story points
- Engineering days
- T-shirt sizes

- Milestone estimates

Large initiatives should be divided into deliverable milestones.

An initiative expected to require more than **12 weeks of engineering work** should normally be decomposed into smaller increments.

---

6. Architecture Standards

a. Architecture Principles

Architecture should be:

- Understandable
- Secure
- Reliable
- Observable
- Maintainable
- Cost-conscious
- Scalable where required

Architecture must reflect actual requirements.

b. Architecture Patterns

Nexora may use:

- Modular monoliths
- Microservices
- Event-driven systems
- Batch processing
- Streaming systems
- Serverless services
- Containerized services
- Managed cloud services

No architecture pattern is mandatory for every product.

c. Architecture Selection

Teams should evaluate:

Consideration	Questions
Scale	How much traffic or data is expected?
Reliability	What failure level is acceptable?

Consideration	Questions
Cost	What is the expected operating cost?
Team	Can the team operate the architecture?
Security	What controls are required?
Complexity	Is the additional complexity justified?
Integration	What systems must connect?

d. Architecture Review

Formal architecture review may be required for:

- Customer-critical systems
- Shared platforms
- Identity systems
- Financial systems
- Security-sensitive systems
- Major cloud migrations
- High-cost infrastructure
- New enterprise-wide technologies

The Technology Architecture Council may request additional documentation.

---

7. Architecture Decision Records

a. Purpose

Architecture Decision Records document important technical decisions.

An ADR should normally contain:

- Decision ID
- Date
- Context
- Problem
- Options
- Decision
- Rationale
- Consequences
- Owner

b. Example

```
ADR-026

Decision:
Use PostgreSQL for the Customer Billing Service.

Alternatives:
1. PostgreSQL
2. MySQL
3. Document database

Reason:
The system requires transactional consistency,
relational reporting and established operational support.

Decision Owner:
Platform Architecture

Date:
12 May 2026
```

c. ADR Review

Architecture decisions should be revisited when assumptions materially change.

An ADR should not be changed silently to rewrite historical decisions.

---

8. Source Control & Repository Management

a. Git Standard

Git is the standard version-control system.

All production software must be maintained in an approved repository.

Repositories should contain:

- Source code
- Tests
- Documentation
- Dependency definitions
- Build instructions
- Configuration templates

b. Repository Naming

Recommended names are:

```
customer-profile-service
payment-processing-api
analytics-ingestion
nexora-assist-api
identity-platform
```

Repository names should be descriptive and stable.

c. Branch Naming

Recommended patterns:

```
feature/<description>
bugfix/<description>
hotfix/<description>
chore/<description>
refactor/<description>
release/<version>
```

Examples:

```
feature/customer-export
bugfix/token-expiry
hotfix/payment-timeout
```

d. Protected Branches

Production branches must use repository protection controls.

At minimum, protected production branches should require:

- Pull request
- Required review
- Successful CI
- No unresolved mandatory checks

Direct pushes to protected production branches are prohibited unless an approved emergency process applies.

---

9. Commit & Pull Request Standards

a. Commit Standards

Commits should describe meaningful changes.

Recommended prefixes include:

```
feat:
fix:
refactor:
test:
docs:
chore:
perf:
security:
```

Example:

```
feat: add customer transaction export
```

b. Pull Requests

Every production-bound change should normally use a pull request.

A pull request should contain:

- Summary
- Business or technical reason
- Testing performed
- Configuration changes
- Security implications
- Deployment considerations

c. Review Requirements

Repository Type	Minimum Approval
Standard	1
Critical	2
Security-sensitive	2 + Security where required

Authors may not approve their own pull requests.

d. Pull Request Size

The recommended size is approximately **500 changed lines or fewer**, excluding generated files.

Large changes should be split where practical.

e. Review Time

Teams should aim to provide an initial review response within **1 business day**.

Urgent production fixes may follow an expedited process.

10. Coding Standards

a. General Standards

Engineers should write code that is:

- Readable
- Testable
- Modular
- Consistent
- Explicit
- Maintainable

Code should favor clarity over cleverness.

b. Naming

Names should describe purpose.

Poor:

```
x = get_data()
```

Preferred:

```
customer_transactions = get_customer_transactions()
```

c. Functions

Functions should have focused responsibilities.

As a general guideline:

- Functions below 50 lines are preferred.
- Functions above 100 lines should be reviewed for decomposition.
- Deep nesting should be avoided.

These are guidelines rather than absolute limits.

d. Error Handling

Errors must not be silently ignored.

The system should:

- Detect the failure

- Record useful diagnostic information
- Return an appropriate response
- Recover where possible

e. Configuration

Environment-specific values should be externalized.

Examples include:

- URLs
- Database names
- Feature flags
- Service configuration
- Credentials

---

11. Software Testing

a. Testing Philosophy

Testing is a shared engineering responsibility.

Testing should occur throughout development rather than only before release.

The standard test layers are:

```
Unit
  ↓
Integration
  ↓
System
  ↓
End-to-End
```

b. Unit Testing

Unit tests should cover:

- Business logic
- Validation
- Calculations
- Error conditions
- Boundary cases

New production code should target at least **80% unit-test coverage**.

Critical services should target **90% or higher**.

c. Integration Testing

Integration testing should validate interactions with:

- Databases
- APIs
- Queues
- External systems
- Authentication services

d. End-to-End Testing

End-to-end testing should cover critical customer journeys.

Teams should avoid excessive end-to-end tests because they are slower and more expensive to maintain.

e. Regression Testing

Regression suites should protect critical existing functionality.

Regression testing is especially important after changes to:

- Shared libraries
- Authentication
- Payments
- APIs
- Data pipelines
- Core workflows

---

12. Quality Engineering

a. Quality Ownership

Quality Engineering works with product and engineering teams to establish:

- Test strategies
- Automation
- Defect prevention
- Regression coverage
- Quality metrics

Quality is not solely the responsibility of QA.

b. Defect Severity

Severity	Description
Sev 1	Critical outage or severe customer impact
Sev 2	Major functionality unavailable
Sev 3	Moderate functional issue
Sev 4	Minor or cosmetic issue

c. Defect Management

Defects should contain:

- Description
- Environment
- Steps to reproduce
- Expected result
- Actual result
- Severity
- Priority
- Owner
- Evidence where appropriate

d. Escaped Defects

A defect discovered in production after release is considered an escaped defect.

Teams should analyze significant escaped defects to determine whether:

- Requirements were unclear
- Tests were insufficient
- Review missed the issue
- Deployment introduced the problem
- Monitoring failed

---

13. API Engineering

a. API Design

APIs should have clearly defined contracts.

Documentation should specify:

- Endpoint
- Method

- Authentication
- Request
- Response
- Errors
- Rate limits
- Version

b. HTTP Standards

Method	Purpose
GET	Retrieve
POST	Create or trigger
PUT	Replace
PATCH	Partial update
DELETE	Remove

c. Response Standards

API responses should be consistent.

Example:

```
{
  "data": {
    "customer_id": "CUST-10482",
    "status": "active"
  },
  "request_id": "REQ-2026-08412"
}
```

Error example:

```
{
  "error": {
    "code": "INVALID_REQUEST",
    "message": "Customer ID is required"
  },
  "request_id": "REQ-2026-08413"
}
```

Internal stack traces must not be returned to external users.

d. API Versioning

Breaking changes should use versioning or an approved migration strategy.

Customers and internal consumers should receive appropriate notice before a breaking change.

---

14. Database Engineering

a. Database Selection

Database selection should consider:

- Data model
- Consistency requirements
- Query patterns
- Scale
- Availability
- Cost
- Operational capability

b. Schema Design

Production schemas should use:

- Appropriate primary keys
- Foreign keys where appropriate
- Valid data types
- Constraints
- Appropriate indexes

c. Query Performance

Queries should be evaluated for:

- Full-table scans
- Expensive joins
- Missing indexes
- Unnecessary columns
- High-frequency access

d. Database Changes

Schema changes must be version controlled through migrations.

Production changes should not rely on undocumented manual database modifications.

e. Data Access

Applications should use parameterized queries or approved ORM/database access mechanisms.

Credentials must never be hard-coded.

---

15. Frontend Engineering

a. Frontend Principles

Frontend applications should provide:

- Clear navigation
- Responsive layouts
- Accessible controls
- Predictable error handling
- Appropriate loading states
- Consistent visual behavior

b. Browser Compatibility

Products should define supported browsers based on customer requirements.

c. Accessibility

User-facing applications should consider:

- Keyboard navigation
- Screen readers
- Color contrast
- Form labels
- Error messages
- Focus management

d. Performance

Frontend teams should monitor:

- Initial load time
- JavaScript bundle size
- API latency
- Rendering performance
- Asset size

Performance targets should be established for customer-facing products where relevant.

---

16. Backend Engineering

a. Service Design

Backend services should separate:

- Transport/API layer
- Business logic
- Data access
- Configuration
- External integrations

The exact architecture may differ by language and framework.

b. Validation

Services must validate:

- Input types
- Required fields
- Ranges
- Authorization
- Business rules

Validation must occur server-side even when frontend validation exists.

c. Idempotency

Operations that may be retried should use idempotency mechanisms where duplicate execution could cause financial or business impact.

Examples include:

- Payments
- Orders
- Customer creation
- Provisioning

---

17. Event-Driven & Asynchronous Systems

a. Event Standards

Events should define:

- Event name
- Producer

- Consumer
- Schema
- Version
- Timestamp
- Unique identifier

Example:

```
{
  "event_id": "EVT-928144",
  "event_type": "customer.updated",
  "version": "2",
  "timestamp": "2026-06-15T09:40:00Z",
  "customer_id": "CUST-10482"
}
```

b. Message Processing

Consumers should consider:

- Duplicate events
- Ordering
- Retries
- Dead-letter handling
- Poison messages
- Schema changes

c. Retry Strategy

Retries should use controlled backoff.

Unbounded retry loops are prohibited.

---

18. Cloud-Native Engineering

a. Cloud Principles

Cloud workloads should be designed for:

- Automation
- Observability
- Security
- Elasticity
- Cost control
- Recoverability

Nexora primarily uses approved Google Cloud capabilities for its cloud-native workloads, subject to customer and project requirements.

b. Infrastructure as Code

Production infrastructure should use approved infrastructure-as-code practices where practical.

Infrastructure code should be:

- Version controlled
- Reviewed
- Tested
- Traceable

c. Containers

Containerized services should:

- Use approved base images
- Minimize unnecessary packages
- Run with appropriate privileges
- Avoid embedded credentials
- Be scanned for vulnerabilities

---

19. CI/CD

a. Continuous Integration

Every significant code change should trigger automated validation.

A standard pipeline may include:

```
Commit
 ↓
Dependency Install
 ↓
Formatting
 ↓
Linting
 ↓
Unit Tests
 ↓
Coverage
 ↓
Security Scan
 ↓
Build
 ↓
Integration Tests
 ↓
Artifact
```

The target for a standard CI pipeline is **15 minutes or less**.

b. Continuous Delivery

Approved artifacts should be deployable without rebuilding source code.

Production deployment should use an approved pipeline.

c. Build Reproducibility

Builds should be reproducible.

Dependencies should be pinned or controlled appropriately.

d. Pipeline Security

CI/CD systems must protect:

- Credentials
- Signing keys
- Deployment tokens
- Artifact repositories
- Production environments

---

20. Release Engineering

a. Release Versioning

Products should use a consistent versioning scheme appropriate to the technology.

A typical version might be:

```
3.8.2
```

where major, minor and patch changes have defined meanings.

b. Release Candidate

A release candidate should contain:

- Version
- Included changes
- Test results
- Security status
- Known issues
- Deployment plan
- Rollback plan

c. Release Approval

Production releases require appropriate approval based on system criticality.

Critical systems may require:

- Engineering approval
- Product approval
- Operations approval
- Security approval where applicable

d. Deployment Record

Production deployments must record:

- Version
- Commit
- Date
- Time
- Environment
- Deployer
- Change reference

---

21. Deployment Strategies

a. Standard Deployment

The new version replaces the previous version.

Appropriate for low-risk systems.

b. Rolling Deployment

Instances are updated progressively.

This may reduce service interruption.

c. Canary Deployment

A limited percentage of traffic is routed to the new release.

Example:

```
1% → 5% → 25% → 50% → 100%
```

Progression should depend on observed health.

d. Blue-Green Deployment

Two environments are maintained.

Traffic is moved from the current version to the new version after validation.

e. Feature Flags

Features may be deployed without immediately enabling them for all users.

Feature flags must have:

- Owner
- Purpose
- Creation date
- Default state
- Removal target

---

22. Production Reliability

a. Reliability Principles

Production software should be:

- Observable
- Recoverable
- Monitored
- Owned
- Documented

b. Service Targets

Typical engineering reference targets include:

Metric	Reference Target
Standard API availability	≥99.9%
Standard API p95 latency	≤500 ms
Critical API p95 latency	≤300 ms
Background processing success	≥99%
Standard API error rate	<1%

Customer contracts may establish stricter requirements.

c. Health Checks

Services should expose appropriate health information.

Health checks may cover:

- Application state
- Database connectivity
- Dependency health
- Queue connectivity

Health checks must not expose sensitive information.

---

23. Observability & Logging

a. Observability

Production systems should provide:

- Logs
- Metrics
- Traces where appropriate
- Dashboards
- Alerts

b. Required Operational Signals

Teams should monitor:

- Request volume
- Error rate
- Latency
- Resource usage
- Dependency failures

c. Logging

A standard production log should include:

- Timestamp
- Service
- Environment
- Severity
- Request ID
- Operation
- Error details where relevant

d. Sensitive Information

The following must not be logged:

- Passwords
- Access tokens
- Private keys
- Full payment-card numbers
- Authentication secrets
- Restricted personal information

e. Log Retention

Reference retention periods are:

Log Type	Retention
Development	7 days
Standard Production	30 days
Security-Relevant	90 days

Customer or regulatory requirements may require longer retention.

24. Security Engineering in Software

a. Secure Development

Engineering teams must incorporate:

- Authentication
- Authorization
- Input validation
- Secure configuration
- Dependency management
- Secret management
- Security testing

b. Authentication

Authentication must use approved identity mechanisms.

Passwords must not be stored in plaintext.

c. Authorization

Authorization should follow least privilege.

The system should validate that the authenticated user is permitted to perform the requested operation.

d. Dependency Security

Dependency vulnerabilities should be remediated according to severity.

Severity	Target
Critical	7 days
High	14 days
Medium	30 days
Low	90 days

Critical internet-facing vulnerabilities may require immediate mitigation.

---

25. Secrets & Configuration Management

a. Secrets

Secrets include:

- Passwords
- API keys
- Tokens
- Private keys
- Database credentials

Secrets must be stored in approved secret-management systems.

b. Source Control

Secrets must never be committed to source control.

If a secret is accidentally committed:

1. Access must be restricted where possible.
2. The credential must be rotated.
3. The repository history should be assessed.
4. Security must be notified where required.

c. Rotation

Service credentials should normally be rotated at least every **90 days** where supported.

Compromised credentials must be rotated immediately.

---

26. Dependency & Open-Source Management

a. Approved Dependencies

Teams should evaluate:

- Security
- License
- Maintenance
- Community activity
- Compatibility
- Cost

b. License Review

Licenses with significant legal or distribution implications require Legal review.

c. Dependency Inventory

Production systems should maintain a dependency inventory where technically practical.

d. Unsupported Components

Teams should create remediation plans for unsupported frameworks or libraries.

A production system relying on a component with no security support should be considered a technology risk.

---

27. Technical Documentation

a. Repository README

Production repositories should document:

- Purpose
- Architecture
- Setup
- Configuration
- Testing
- Deployment
- Ownership
- Support

b. API Documentation

APIs should maintain current contracts.

c. Runbooks

Operationally significant systems should have runbooks covering:

- Common failures
- Recovery
- Restart
- Rollback
- Escalation

d. Architecture Documentation

Significant systems should maintain:

- Architecture overview
- Major dependencies
- Data flows
- External integrations

---

28. Technical Debt

a. Definition

Technical debt is the future engineering cost created by decisions that reduce maintainability, reliability or scalability.

Examples include:

- Outdated libraries
- Temporary workarounds
- Duplicate implementations
- Manual processes
- Missing tests
- Incomplete documentation

b. Management

Technical debt should be recorded and prioritized.

Each significant item should include:

- Description
- Impact
- Risk
- Owner
- Priority
- Estimated effort

c. Capacity

Engineering teams should consider allocating approximately **10–15% of capacity** toward technical debt and engineering improvement, subject to product priorities.

---

29. Engineering Incidents & Production Defects

a. Production Incident

A production incident occurs when a system experiences an unexpected condition affecting availability, performance, security or business functionality.

b. Response

Engineering teams should:

1. Detect
2. Assess

3. Stabilize
4. Communicate
5. Recover
6. Investigate
7. Prevent recurrence

c. Root Cause Analysis

Significant incidents should result in a post-incident review.

The review should identify:

- What happened
- Impact
- Timeline
- Contributing factors
- Root causes
- Detection gaps
- Corrective actions

The objective is learning and system improvement rather than individual blame.

---

30. Change Failure & Deployment Quality

a. Change Failure Rate

A production change is considered failed when it causes significant:

- Outage
- Degradation
- Customer defect
- Rollback
- Emergency remediation

b. Engineering Threshold

A sustained change-failure rate above **15% for two consecutive monthly reporting periods** should trigger engineering review.

The review should consider:

- Testing
- Review quality
- Deployment process
- Architecture
- Monitoring

- Rollback capability

c. Deployment Frequency

Teams may track deployment frequency as a delivery-health indicator.

Higher deployment frequency is valuable only when quality and reliability remain acceptable.

---

31. Performance Engineering

a. Performance Requirements

Performance requirements should be established before implementation when performance is business-critical.

b. Load Testing

Load tests may evaluate:

- Concurrent users
- Requests per second
- Response time
- CPU
- Memory
- Database load

c. Capacity Planning

Teams should monitor capacity trends and plan before resource exhaustion.

Capacity planning may consider:

- Current usage
- Growth rate
- Seasonal demand
- Customer growth
- Infrastructure limits

d. Performance Regression

A significant performance regression should be treated as a defect when it violates an approved performance requirement.

---

32. Accessibility & User Experience

a. Accessibility

Customer-facing products should aim to meet applicable accessibility requirements.

Teams should consider:

- Keyboard access
- Screen-reader compatibility
- Focus states
- Form labels
- Error messages
- Contrast
- Text scaling

b. User Experience

Product and engineering teams should work together to ensure that:

- Workflows are understandable
- Errors are actionable
- Loading states are clear
- Destructive actions require appropriate confirmation

---

33. Mobile & Client Applications

Where Nexora develops mobile or client applications, teams must consider:

- Supported operating systems
- Version compatibility
- Secure storage
- Authentication
- Network failures
- Offline behavior
- Application updates

Sensitive information should not be stored locally without appropriate protection.

---

34. AI-Enabled Software

a. AI Development

AI-enabled products must follow applicable AI governance standards.

Teams should document:

- Model/provider
- Intended purpose
- Data sources
- Evaluation
- Known limitations
- Security controls
- Human oversight
- Cost

b. Generative AI

Generative AI systems should be evaluated for:

- Accuracy
- Grounding
- Hallucination
- Prompt injection
- Data leakage
- Toxic or unsafe outputs
- Latency
- Cost

c. Production Evaluation

AI systems should have measurable acceptance criteria.

Examples:

Metric	Example
Task success	≥90%
Grounded responses	≥95%
Critical safety failures	0
Median response time	≤5 seconds

Actual targets depend on use case.

35. Data Engineering in Product Development

a. Data Pipelines

Production data pipelines should have:

- Defined owners
- Source documentation
- Schema definitions
- Validation
- Monitoring
- Failure handling

b. Data Quality

Teams should monitor:

- Completeness
- Accuracy
- Timeliness
- Uniqueness
- Validity

c. Data Contracts

Shared data producers and consumers should document important schema expectations.

Breaking schema changes require coordinated migration.

---

36. Developer Experience

a. Development Environment

Engineering teams should provide standardized development guidance.

A new engineer should be able to understand:

- Repository structure
- Local setup
- Required dependencies
- Testing
- Build
- Deployment process

b. Onboarding

Engineering onboarding should include:

- Repository access
- Development environment
- Security training
- Architecture overview
- Team processes
- Code-review practices

The engineering manager should ensure the employee has sufficient access to begin productive work.

c. Developer Productivity

Teams may measure:

- Build time
- CI duration
- Environment setup time
- Deployment frequency
- Review time
- Developer satisfaction

Metrics should be used for improvement rather than employee surveillance.

---

37. Product Quality Metrics

Product teams may monitor:

Metric	Purpose
Defect rate	Product quality
Escaped defects	Release quality
Availability	Reliability
Latency	Performance
Adoption	Product value
Retention	Customer value
Support tickets	User friction
Customer satisfaction	Experience
Release frequency	Delivery capability

No single metric should be optimized in isolation.

38. Engineering Governance

a. Engineering Standards

Engineering standards are maintained by the Technology organization.

Standards are reviewed at least annually or earlier when significant technology changes occur.

b. Exceptions

Exceptions should include:

- Standard being waived
- Reason
- Risk
- Compensating controls
- Owner
- Expiry date

c. Exception Review

Permanent exceptions should be reviewed annually.

Security, legal and regulatory requirements cannot be overridden through ordinary engineering exceptions.

39. Software Lifecycle & Retirement

a. Lifecycle States

Software systems generally progress through:

```
Concept
→ Development
→ Production
→ Maintenance
→ Modernization
→ Retirement
```

b. Lifecycle Review

Systems should be reviewed when:

- Technology becomes unsupported
- Operating cost increases significantly

- Incident frequency increases
- Customer adoption declines
- Security risk increases
- A replacement becomes available

c. Retirement

Retirement should include:

- Stakeholder communication
- Data retention assessment
- Access removal
- Infrastructure shutdown
- Dependency removal
- Documentation update
- Repository archival

---

40. Engineering Metrics & Operating Review

a. Delivery Metrics

Engineering leadership may review:

- Lead time
- Cycle time
- Deployment frequency
- Sprint completion
- Review turnaround

b. Quality Metrics

Metrics may include:

- Automated coverage
- Defect rate
- Escaped defects
- Change failure rate
- Production incidents

c. Reliability Metrics

Metrics may include:

- Availability
- Error rate

- Latency
- Mean time to detect
- Mean time to recover

d. Security Metrics

Metrics may include:

- Vulnerability count
- Vulnerability age
- Critical findings
- Secret incidents
- Security exceptions

e. Review Cadence

Engineering health should normally be reviewed at least monthly at team level and quarterly at broader organizational level.

---

41. Example Product Architecture

The following example illustrates a typical Nexora customer application.



Supporting services may include:

- Identity
- Notification
- Audit
- Monitoring
- Configuration

The exact architecture depends on requirements.

---

42. Example Engineering Project

a. Project

Project ID: NXR-PRD-0427
Project: Customer Analytics Platform
Business Unit: Data & AI
Engineering Lead: Assigned by Technology
Target Release: Q3 FY2026

b. Scope

The platform provides customer analytics through:

- Data ingestion
- Transformation
- Analytics APIs
- Dashboards
- Role-based access

c. Engineering Targets

Metric	Target
API availability	99.9%
API p95 latency	<500 ms
New-code coverage	≥85%
Critical vulnerabilities	0
Standard pipeline duration	<15 min
Production deployment	Fully traceable

d. Architecture

The platform uses:

- Cloud-hosted services
- PostgreSQL
- Data warehouse
- REST APIs
- Containerized services
- CI/CD

- Centralized identity

e. Release

Version `1.0.0` is released after:

- Unit testing
- Integration testing
- Security validation
- Staging validation
- Product approval
- Engineering approval

---

43. Engineering Checklists

a. Before Development

- [ ] Problem understood
- [ ] Requirements documented
- [ ] Acceptance criteria defined
- [ ] Architecture reviewed
- [ ] Security requirements identified
- [ ] Dependencies identified
- [ ] Owner assigned

b. Before Merge

- [ ] Code follows standards
- [ ] Tests added
- [ ] Tests pass
- [ ] Coverage meets target
- [ ] Security checks pass
- [ ] Documentation updated
- [ ] Pull request reviewed
- [ ] Required approvals obtained

c. Before Production

- [ ] Release candidate validated
- [ ] Regression tests pass
- [ ] Security checks complete
- [ ] Monitoring available
- [ ] Deployment record prepared

- [ ] Rollback strategy confirmed
- [ ] Stakeholders informed
- [ ] Production owner confirmed

d. After Production

- [ ] Health verified
- [ ] Error rates checked
- [ ] Latency checked
- [ ] Business metrics checked
- [ ] Logs reviewed
- [ ] Deployment closed
- [ ] Follow-up actions recorded

---

44. Reference Standards & Related Documents

This handbook should be read together with:

- **NXR-CORP-001** — Company, People & Corporate Governance Handbook
- **NXR-ENG-STD-003** — Production Change Management
- **NXR-SEC-POL-001** — Information Security Policy
- **NXR-SEC-STD-002** — Information Classification Standard
- **NXR-SEC-IRP-003** — Security Incident Response Plan
- **NXR-DATA-STD-001** — Data Governance Standard
- **NXR-DATA-STD-002** — Analytics & Reporting Standards
- **NXR-AI-POL-001** — Responsible AI & Generative AI Policy
- **NXR-AI-STD-002** — AI Development & Evaluation
- **NXR-DEL-STD-001** — Project Delivery Framework
- **NXR-DEL-PRC-002** — Customer Escalation Procedure
- **NXR-OPS-STD-001** — Business Continuity Standard

Where another approved standard establishes a stricter requirement, the stricter requirement applies unless an authorized exception exists.

Appendix A — Engineering Standards Summary

Area	Standard / Target
Source control	Git
Standard PR approvals	1
Critical PR approvals	2

Area	Standard / Target
Review response target	≤1 business day
Recommended PR size	≤500 changed lines
New production code coverage	≥80%
Critical service coverage	≥90%
CI pipeline target	≤15 minutes
Critical vulnerability remediation	≤7 days
High vulnerability remediation	≤14 days
Medium vulnerability remediation	≤30 days
Standard production log retention	30 days
Security log retention	90 days
Secret rotation target	90 days
Standard API availability target	≥99.9%
Standard API p95 latency	≤500 ms
Critical API p95 latency	≤300 ms
Change-failure review trigger	>15% for 2 months
Technical debt guideline	10–15% capacity

Appendix B — Example Repository Structure

```
customer-profile-service/
│
├── src/
│   ├── api/
│   ├── services/
│   ├── models/
│   ├── repositories/
│   └── utils/
│
├── tests/
│   ├── unit/
│   ├── integration/
│   └── e2e/
│
├── docs/
│   ├── architecture/
│   ├── api/
│   └── runbooks/
│
├── scripts/
│
├── Dockerfile
├── README.md
├── requirements.txt
├── pyproject.toml
└── .gitignore
```

The structure may vary by programming language.

Appendix C — Example Pull Request Template

```
Title:
[FEATURE] Customer transaction export

Summary:
Adds CSV export capability for authorized finance users.

Business Reason:
Reduce manual monthly reconciliation work.

Changes:
- Added export API
- Added authorization checks
- Added CSV generation
- Added audit logging

Testing:
- Unit tests: 18
- Integration tests: 6
- Coverage: 87%

Security:
- Authorization validated
- No restricted data written to logs

Deployment:
Requires application version 2.7.0.

Rollback:
Revert to 2.6.4.
```

Appendix D — Example Release Record

```
Release ID: REL-2026-0714
Application: Customer Analytics Platform
Version: 1.4.0
Environment: Production
Commit: 8f21c4d
Release Owner: Data Platform Engineering
Change Reference: CHG-2026-0481

Unit Tests: Passed
Integration Tests: Passed
Security Scan: Passed
Staging Validation: Passed

Deployment Start:
14 July 2026, 21:00 IST

Deployment Complete:
14 July 2026, 21:17 IST

Post-Deployment Validation:
Passed

Rollback Version:
1.3.2
```

Appendix E — Engineering Decision Matrix

Decision	Primary Owner	Additional Review
Programming language	Technical Lead	Architecture if new technology
Framework	Technical Lead	Architecture if enterprise-wide
Database	Technical Lead	Architecture for critical systems
Cloud service	Platform/Architecture	Security where applicable
External SaaS	Engineering + Procurement	Security/Legal
Production architecture	Architecture	Technology Architecture Council where required
Security architecture	Security	CISO or delegate
Customer-facing breaking API	Engineering + Product	Delivery/Customer team
Major AI model	AI Engineering	AI Governance Committee
Restricted data processing	Data Owner	Security/Privacy

Appendix F — Key Engineering Terms

Availability
The proportion of time a service remains operational and accessible.

Change Failure Rate
The proportion of production changes resulting in failure, rollback or significant remediation.

CI/CD
Continuous Integration and Continuous Delivery/Deployment practices used to automate software validation and release.

Technical Debt
Future engineering cost resulting from shortcuts, temporary solutions or outdated technology.

Pull Request
A controlled request to merge source-code changes into another branch.

SLO
Service Level Objective defining a target for a service characteristic such as availability or latency.

Rollback
Returning a production system to a previous known-good version or state.

Feature Flag
A mechanism that allows functionality to be enabled or disabled independently of deployment.

Architecture Decision Record
A document capturing an important technical decision and its reasoning.

Production
The environment serving live users, customers or business operations.

Document Approval

Document Owner: Technology Organization
Executive Sponsor: Vikram Rao, Chief Technology Officer
Effective Date: 01 April 2026
Review Date: 31 March 2027
Classification: Internal

End of Document